

OpenClaw Peripheral Hardware Configuration

OpenClaw Peripheral Hardware Configuration

1. Course Overview
2. Hardware Configuration File Description
 - 2.1 Configuration File Location
 - 2.2 Current Configuration File Coverage
3. DHT Hardware Testing
 - 3.1 Verification Method
4. RGB Hardware Testing
 - 4.1 Verification Method
5. Servo Hardware Testing
 - 5.1 Verification Method
6. OpenClaw Firmware Serial Port Settings
 - 6.1 Current Default Values
 - 6.2 Modification Example
 - 6.3 Verification Method
7. OLED Hardware Testing
 - 7.1 Current Default Values
 - 7.2 Scenarios Suitable for Modification
 - 7.3 Common Modification Items
 - Modify Resolution
 - Modify I2C Address
 - Modify Rotation Direction
 - 7.4 Verification Method
8. Camera Hardware Testing
 - 8.1 Configuration
 - 8.2 Check Ports
 - 8.3 Photo Capture Verification Method
 - 8.4 Adjustable Parameter Description

1. Course Overview

This course will introduce the default configuration files related to hardware before using OpenClaw, focusing on how to modify the underlying default parameters according to your actual peripheral situation and complete basic verification.

- Use low-level commands to verify if modifications take effect

2. Hardware Configuration File Description

2.1 Configuration File Location

OpenClaw's current hardware default configuration file is located at:

```
/home/pi/hardware_ha1_demo/hardware_config.py
```

The default uses the factory image. If you are only following the tutorial and using the factory hardware, you usually don't need to modify it.

2.2 Current Configuration File Coverage

`hardware_config.py` currently mainly includes three types of default configurations:

- OpenClaw firmware serial port default parameters
- OLED default parameters

These default values are read by low-level command line tools, such as:

- `rgb`
- `oled`
- `dht`

Important notes:

- DHT and RGB are driven directly through the OpenClaw expansion board firmware
- OLED is driven through the motherboard's I2C pins
- CSI camera needs to check hardware connection

3. DHT Hardware Testing

Note: If you need to continuously read DHT through the Python driver library, it is recommended to read DHT data at a two-second interval, as DHT sensors are more suitable for 2s sampling

3.1 Verification Method

```
dht      #Get sensor data
dht_oled  #Display sensor data on OLED
```

Normal terminal display will show temperature and humidity data on OLED

```
pi@raspberrypi:~$ dht
{"ok": true, "sensor": "dht11", "temperature_c": 27.2, "temperature_f": 81.0, "humidity": 56.0, "attempts": 1}
pi@raspberrypi:~$ dht_oled
{"ok": true, "code": "ok", "stage": "completed", "message": "温湿度已显示到 OLED", "reading": {"ok": true, "sensor": "dht11", "temperature_c": 27.2, "temperature_f": 81.0, "humidity": 57.0, "attempts": 1}, "oled": {"title": "DHT11", "text": "Temp:27.2C\\nHum :57.0%", "lines": ["DHT11", "Temp:27.2C", "Hum :57.0%"], "dry_run": false, "i2c_bus": "auto", "render_title": "DHT11", "render_text": "Temp:27.2C\\nHum :57.0%"/>
```

4. RGB Hardware Testing

4.1 Verification Method

Run in terminal:

```
rgb color --color red #All turn red
rgb effect --effect breathe --color blue --speed 5 #Blue breathing light
```

Normal terminal display will show RGB lights displaying normally

```
pi@raspberrypi:~$ rgb color --color red
{"ok": true, "code": "ok", "message": "RGB 颜色已设置", "details": {"index": 255, "r": 255, "g": 0, "b": 0}}
pi@raspberrypi:~$ rgb effect --effect breathe --color blue --speed 5
{"ok": true, "code": "ok", "message": "RGB 特效已设置: breathe", "details": {"effect": "breathe", "speed": 5, "color": "blue"}}
```

Turn off the light strip

```
rgb off
```

```
pi@raspberrypi:~ $ rgb off
{"ok": true, "code": "ok", "message": "RGB 颜色已设置", "details": {"index": 255, "r": 0, "g": 0, "b": 0}}
pi@raspberrypi:~ $
```

5. Servo Hardware Testing

5.1 Verification Method

Control single servo:

```
servo call-tool Servo1 --action set_angle --angle 90 #Servo 1 rotates to 90
degrees
servo call-tool Servo2 --action set_angle --angle 180 #Servo 2 rotates to 90
degrees

#Servo1 is the servo to control
#90 is the angle to rotate to
```

```
params : { "angle": 0.0}, "retryable": false}
pi@raspberrypi:~ $ servo call-tool Servo1 --action set_angle --angle 180
{"ok": true, "code": "ok", "message": "done", "device_id": "servo_1", "action": "set_angle", "request_id": "", "stat
e": {"angle": 180.0, "attached": true, "last_angle": 180.0, "min_angle": 0.0, "max_angle": 180.0}, "telemetry": {"mo
de": "direct", "driver": "openclaw_servo", "servo_id": 1, "min_angle": 0.0, "max_angle": 180.0, "action": "set_angle
", "params": {"angle": 180.0}}, "retryable": false}
pi@raspberrypi:~ $ servo call-tool Servo2 --action set_angle --angle 180
{"ok": true, "code": "ok", "message": "done", "device_id": "servo_2", "action": "set_angle", "request_id": "", "stat
e": {"angle": 180.0, "attached": true, "last_angle": 180.0, "min_angle": 0.0, "max_angle": 180.0}, "telemetry": {"mo
de": "direct", "driver": "openclaw_servo", "servo_id": 2, "min_angle": 0.0, "max_angle": 180.0, "action": "set_angle
", "params": {"angle": 180.0}}, "retryable": false}
pi@raspberrypi:~ $
```

Synchronously control multiple servos:

Note: If there are only two servos, fill in the angles for both servos, the angles for the other two servos can be arbitrary

```
# Four servos simultaneously reach specified angles
servo call-tool ServoAll --action set_all --angle1 90 --angle2 45 --angle3 135 --
angle4 60
# All servos return to center position
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --
angle4 90
```

6. OpenClaw Firmware Serial Port Settings

6.1 Current Default Values

Current OpenClaw firmware serial port default parameters are:

- `port="/dev/ttyAMA0"`
- `baudrate=115200`

Other motherboards' pin serials may not be named this way. For example, Jetson's is `"/dev/ttyTHS1"`

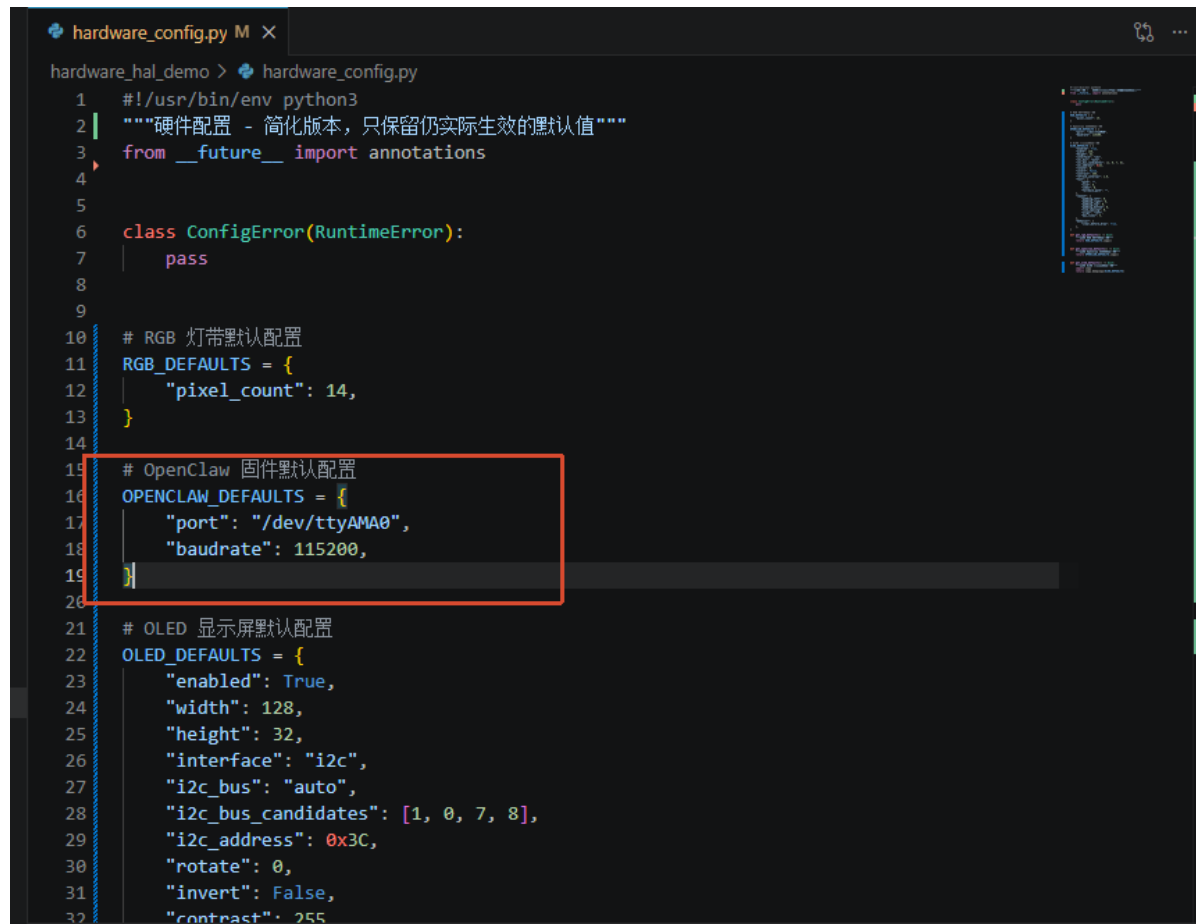
6.2 Modification Example

Edit the configuration file:

```
nano /home/pi/hardware_hal_demo/hardware_config.py
```

Change Raspberry Pi to, for example:

```
"port": "/dev/ttyAMA0",  
"baudrate": 115200,
```



```
hardware_hal_demo > hardware_config.py  
1  #!/usr/bin/env python3  
2  """硬件配置 - 简化版本, 只保留仍实际生效的默认值"""  
3  from __future__ import annotations  
4  
5  
6  class ConfigError(RuntimeError):  
7      pass  
8  
9  
10 # RGB 灯带默认配置  
11 RGB_DEFAULTS = {  
12     "pixel_count": 14,  
13 }  
14  
15 # OpenCLAW 固件默认配置  
16 OPENCLAW_DEFAULTS = {  
17     "port": "/dev/ttyAMA0",  
18     "baudrate": 115200,  
19 }  
20  
21 # OLED 显示屏默认配置  
22 OLED_DEFAULTS = {  
23     "enabled": True,  
24     "width": 128,  
25     "height": 32,  
26     "interface": "i2c",  
27     "i2c_bus": "auto",  
28     "i2c_bus_candidates": [1, 0, 7, 8],  
29     "i2c_address": 0x3C,  
30     "rotate": 0,  
31     "invert": False,  
32     "contrast": 255
```

6.3 Verification Method

After saving, it is recommended to first test with either servo or RGB low-level commands, for example:

```
rgb color --color green
```

Or:

```
servo call-tool Servo1 --action center #Servo 1 returns to center 90 degrees  
# All servos return to center position  
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --  
angle4 90
```

If commands can return normally and peripherals can be controlled normally, the serial link is usable.

```
pi@raspberrypi:~$ servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --angle4 90
{"ok": true, "code": "ok", "message": "done", "device_id": "all", "action": "set_all", "request_id": "", "state": {"angles": [90, 90, 90, 90]},
"telemetry": {"mode": "direct", "params": {"angle1": 90.0, "angle2": 90.0, "angle3": 90.0, "angle4": 90.0}}, "retryable": false}
pi@raspberrypi:~$
```

7. OLED Hardware Testing

7.1 Current Default Values

Current OLED default parameters mainly include:

- `width=128`
- `height=32`
- `interface="i2c"`
- `i2c_bus="auto"`
- `i2c_bus_candidates=[1, 0, 7, 8]`
- `i2c_address=0x3C`
- `rotate=0`
- `invert=False`
- `contrast=255`

7.2 Scenarios Suitable for Modification

It is recommended to modify OLED default values in the following situations:

- You are using an OLED with different resolution
- I2C address is not `0x3C`
- Screen orientation needs to be flipped

7.3 Common Modification Items

Modify Resolution

For example, change from:

```
"width": 128,
"height": 32,
```

To:

```
"width": 128,
"height": 64,
```

Modify I2C Address

For example, change from:

```
"i2c_address": 0x3C,
```

To:

```
"i2c_address": 0x3D,
```

Modify Rotation Direction

For example, change from:

```
"rotate": 0,
```

To:

```
"rotate": 180,
```

7.4 Verification Method

After saving, run in terminal:

```
oled text --text 'Hello OpenClaw'  
oled clear
```

If OLED can display normally and clear the screen, the configuration has taken effect.

```
pi@raspberrypi:~$ oled text --text 'Hello OpenClaw'  
{'ok': true, 'code': 'ok', 'mode': 'text', 'message': '文字已显示到 OLED', 'details': {'title': '', 'text': 'Hello OpenClaw', 'lines': ['Hello O  
penClaw'], 'dry_run': false}}  
pi@raspberrypi:~$ oled clear  
{'ok': true, 'code': 'ok', 'mode': 'clear', 'message': 'OLED 已清屏', 'details': {'dry_run': false}}  
pi@raspberrypi:~$
```

8. Camera Hardware Testing

8.1 Configuration

The premise is ensuring a CSI camera is connected. The current CSI camera chain mainly works through the following two entry points:

```
/home/pi/hardware_hat_demo/camera  
/home/pi/.openclaw/workspace/skills/pi-vision/scripts/csi_snap
```

8.2 Check Ports

Input in terminal:

```
ls /dev/video*
```

Normally you can see video0 and video1 devices.

```
pi@raspberrypi:~$ ls /dev/video*  
/dev/video0 /dev/video2 /dev/video22 /dev/video25 /dev/video28 /dev/video30 /dev/video33 /dev/video4 /dev/video7  
/dev/video1 /dev/video20 /dev/video23 /dev/video26 /dev/video29 /dev/video31 /dev/video34 /dev/video5  
/dev/video19 /dev/video21 /dev/video24 /dev/video27 /dev/video3 /dev/video32 /dev/video35 /dev/video6  
pi@raspberrypi:~$
```

8.3 Photo Capture Verification Method

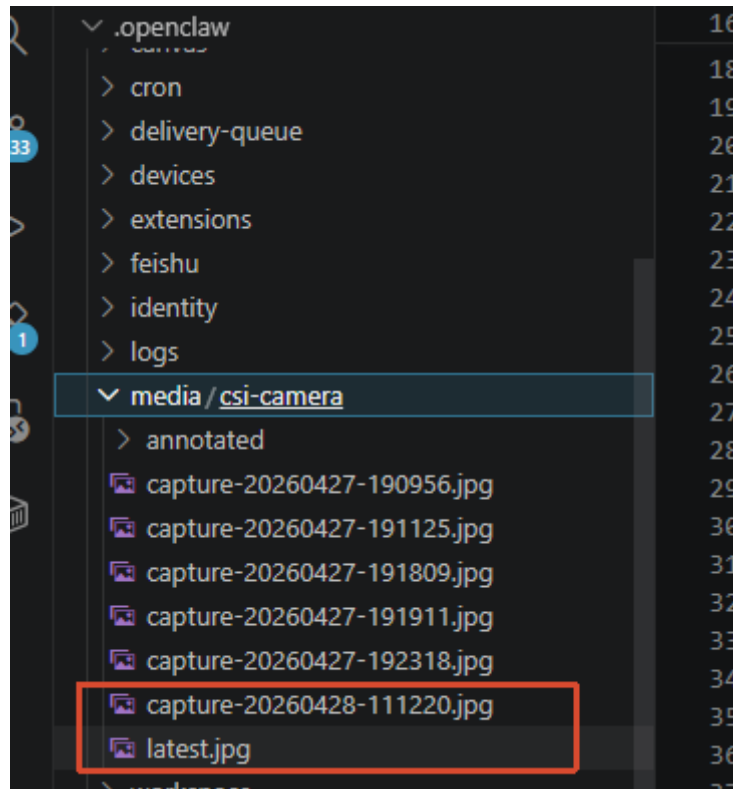
Input in terminal and directly test the photo capture chain:

```
csi_snap snap
```

```
pi@raspberrypi:~$ csi_snap snap
{"ok":true,"code":"ok","mode":"snap","camera_cmd":"rpicam-still","saved_path":"/home/pi/.openclaw/media/csi-camera/capture-20260428-111220.jpg",
"latest_path":"/home/pi/.openclaw/media/csi-camera/latest.jpg","width":640,"height":480,"timeout_ms":1000,"rotation":0,"timestamp":"20260428-11
220","capture_id":"cap-20260428-111220-9234a8414958","sha256":"9234a841495874154de8a8ad2c5cac44ea39487b59e784200ddbe4020cfc640","sha256_12":"92
34a8414958","size_bytes":37795}
pi@raspberrypi:~$
```

If the photo is taken successfully, you will usually see output files in the following directory:

```
~/openclaw/media/csi-camera/
```



8.4 Adjustable Parameter Description

Although camera is currently not configured in `hardware_config.py`, `csi_snap` supports some photo parameters when called, such as:

- `--width`
- `--height`
- `--timeout-ms`
- `--rotation`
- `--hflip`
- `--vflip`
- `--camera`

For example:

```
csi_snap snap --width 1280 --height 720
```

```
pi@raspberrypi:~$ csi_snap snap --width 1280 --height 720
{"ok":true,"code":"ok","mode":"snap","camera_cmd":"rpicam-still","saved_path":"/home/pi/.openclaw/media/csi-camera/capture-20260428-111412.jpg",
"latest_path":"/home/pi/.openclaw/media/csi-camera/latest.jpg","width":1280,"height":720,"timeout_ms":1000,"rotation":0,"timestamp":"20260428-11
1412","capture_id":"cap-20260428-111412-f016435ec011","sha256":"f016435ec011587e7b30aa19e1da9ba09b2ea8d46e5887c3ee2cd6960cda524f","sha256_12":"f
016435ec011","size_bytes":115057}
pi@raspberrypi:~$
```

